Dynamic Programming

Abstract

This topic introduces dynamic programming, a class of algorithms used to solve optimization problems in reinforcement learning. It covers policy evaluation, policy improvement, policy iteration, value iteration, and generalized policy iteration. These sub-topics are fundamental concepts in dynamic programming, providing students with a comprehensive understanding of how dynamic programming can be used to optimize policies in reinforcement learning. By mastering these concepts, students will be prepared for more advanced topics such as Monte Carlo methods and TD-learning.

Learning Objectives

- Implement Policy Evaluation to compute value functions for a given policy
- Apply Policy Improvement to derive better policies from value functions
- Execute Policy Iteration by alternating evaluation and improvement steps
- Implement Value Iteration to directly compute optimal value functions
- Explain the model requirement limitation of DP and why it's unrealistic in practice

Overview of Dynamic Programming Methods

This lecture covers five fundamental dynamic programming methods for reinforcement learning:

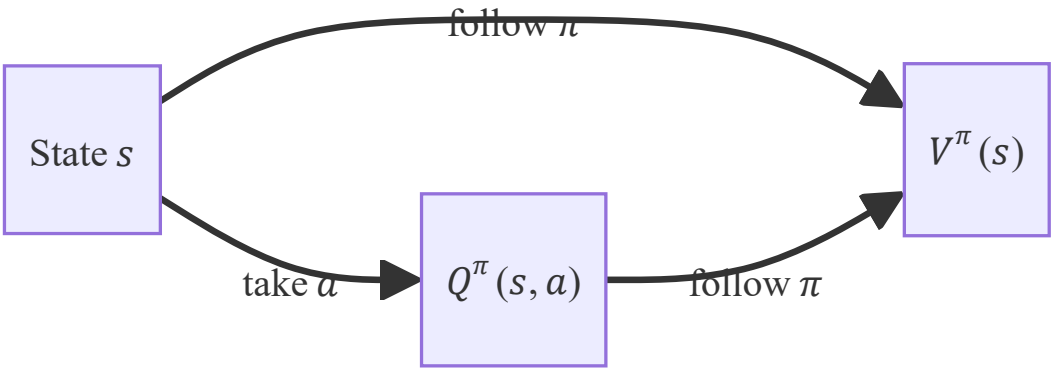
1. **Policy Evaluation** - Computes the value function $V^\pi(s)$ for a given policy using the Bellman expectation operator
2. **Policy Improvement** - Creates better policies by acting greedily with respect to current value estimates
3. **Policy Iteration** - Alternates between policy evaluation and improvement until convergence to optimal policy
4. **Value Iteration** - Directly computes optimal value function $V^*(s)$ using Bellman optimality operator, then extracts policy

These methods form the foundation for understanding how to solve Markov Decision Processes optimally.

Policy Evaluation

- **Why it exists**: We need to measure how good a policy is before we can improve it
- **What it computes**: The value function $V^\pi(s)$ - expected cumulative reward from each state under policy π
- **How it works**: Iteratively applies the Bellman expectation operator until convergence
- **Key insight**: Uses dynamic programming to solve the system of linear equations efficiently

Overview of value functions (rehearsal from L2)



State-value function:

$$V^\pi(s) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s\right]$$

Action-value function:

$$Q^\pi(s,a) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s, a_0 = a\right]$$

Relationship:

$$V_\pi(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}[Q_\pi(s,a)]$$

[(Eysenbach, COS435 Lecture 7, 2025, p.1-4) (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-7-value-functions.pdf)](https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-7-value-functions.pdf)

The Policy Evaluation Algorithm

Bellman expectation operator:

$$V_{k+1}(s) = \mathbb{E}_{a\sim\pi(a|s),s'\sim p(s'|s,a)}[r(s,a) + \gamma V_k(s')]$$

Algorithm:

$$\begin{aligned}
&\textbf{Input: } \pi(\cdot|\cdot), p(\cdot|\cdot,\cdot), r(\cdot,\cdot), \gamma, \theta && (1)\\
&\textbf{Output: } V^{\pi}(\cdot) && (2)\\
& && (3)\\
&\textbf{1. } \text{Initialize } V(s) = 0 \text{ for all } s \in \mathcal{S} && (4)\\
&\textbf{2. Repeat:} && (5)\\
&\quad \textbf{3. } \Delta \leftarrow 0 && (6)\\
&\quad \textbf{4. For each state } s \in \mathcal{S}\textbf{:} && (7)\\
&\quad\quad \textbf{5. } v \leftarrow V(s) && (8)\\
&\quad\quad \textbf{6. } V(s) \leftarrow \sum_a \pi(a|s) \sum_{s'} p(s'|s,a)\left[r(s,a) + \gamma V(s')\right] && (9)\\
&\quad\quad \textbf{7. } \Delta \leftarrow \max(\Delta, |v - V(s)|) && (10)\\
&\textbf{8. Until } \Delta < \theta && (11)\\
&\textbf{9. Return } V && (12)
\end{aligned}$$

[(Eysenbach, COS435 Lecture 8, 2025, p.3-5) (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-8-policy-value-iteration.pdf)](https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-8-policy-value-iteration.pdf)

In [1]:
```python
import numpy as np
import importlib
import helper_functions
importlib.reload(helper_functions)
from helper_functions import *

# Define grid world constants
GRID_SIZE = (3, 3)
NUM_STATES = GRID_SIZE[0] * GRID_SIZE[1]
NUM_ACTIONS = 4

print("✅ Imports complete!")
print(f"Grid world: {GRID_SIZE}, {NUM_STATES} states, {NUM_ACTION
S} actions")

# Define environment functions (for didactic purposes - later mov
ed to helper_functions.py)

def p(s_prime, s, a):
    """Transition probability p(s'|s,a) for deterministic grid wo
rld.
    Actions: 0=up, 1=down, 2=left, 3=right"""
    row, col = divmod(s, GRID_SIZE[1])

    # Calculate next state based on action
    next_state = s  # Default: stay in same state

    if a == 0 and row > 0:  # up
        next_state = (row - 1) * GRID_SIZE[1] + col
    elif a == 1 and row < GRID_SIZE[0] - 1:  # down
        next_state = (row + 1) * GRID_SIZE[1] + col
    elif a == 2 and col > 0:  # left
        next_state = row * GRID_SIZE[1] + (col - 1)
    elif a == 3 and col < GRID_SIZE[1] - 1:  # right
        next_state = row * GRID_SIZE[1] + (col + 1)

    return 1.0 if s_prime == next_state else 0.0


def r(s):
    """Reward function r(s) for state s. Treasure at bottom-right
corner."""
    treasure_state = GRID_SIZE[0] * GRID_SIZE[1] - 1
    return 10.0 if s == treasure_state else -0.1

print("✅ Environment defined!")


def uniform_policy(s, a):
    """Uniform random policy: π(a|s) = 1/4 for all actions"""
    return 1.0 / NUM_ACTIONS
```

```
✅ Imports complete!
Grid world: (3, 3), 9 states, 4 actions
✅ Environment defined!
```

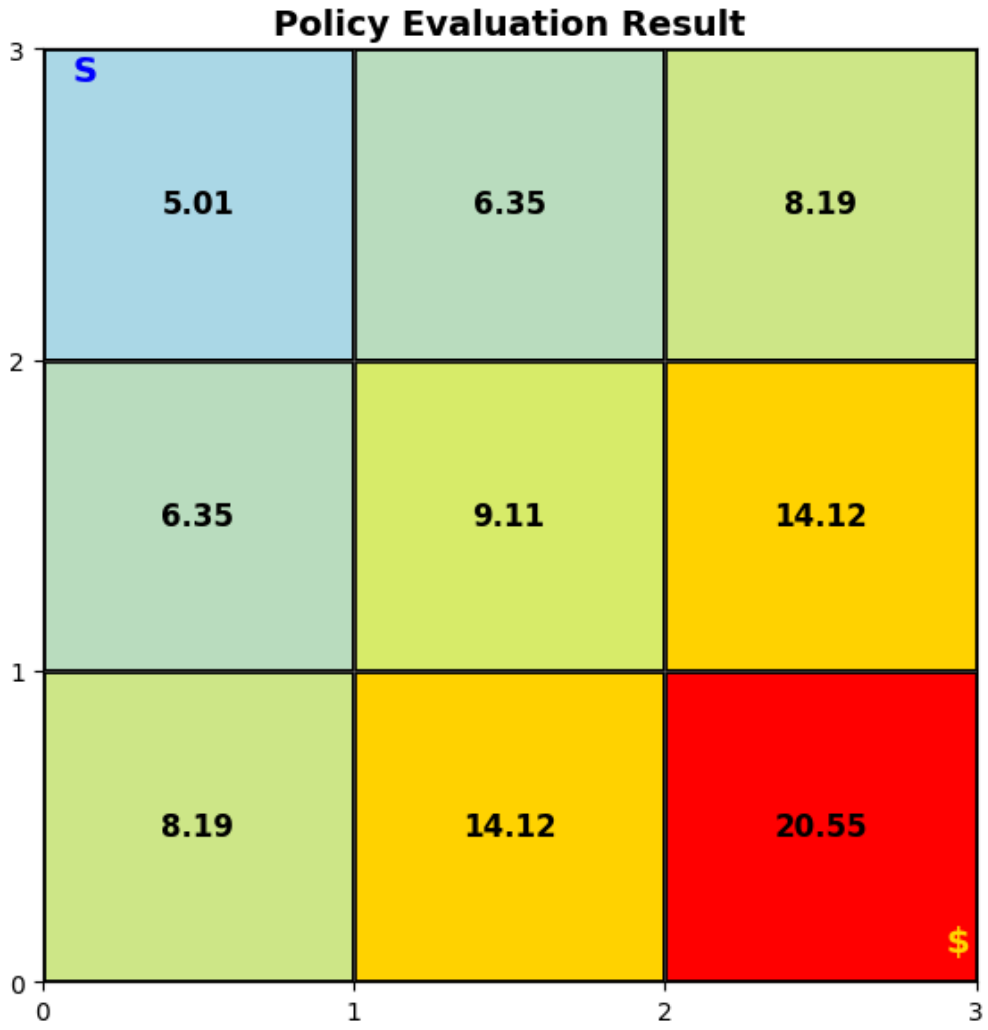
In [2]:
```
# Reimport helpers (for development)
import importlib
import helper_functions
importlib.reload(helper_functions)
from helper_functions import *

# Policy Evaluation Algorithm Implementation

def policy_evaluation_algorithm(policy_func, gamma=0.9, max_itera
tions=100, threshold=0.01):
    """
    Evaluate a policy using iterative policy evaluation.

    Algorithm:
    1. Initialize V(s) = 0 for all states
    2. Repeat until convergence:
        - For each state s:
            - Update V(s) using Bellman expectation equation
        - Check convergence: max|V_old - V_new| < threshold
    """
    V = np.zeros(NUM_STATES)

    for iteration in range(max_iterations):
        delta = 0.0
        V_new = np.zeros(NUM_STATES)

        for s in range(NUM_STATES):
            # Bellman expectation backup
            v = 0.0
            for a in range(NUM_ACTIONS):
                action_value = 0.0
                for s_prime in range(NUM_STATES):
                    action_value += p(s_prime, s, a) * (r(s_prim
e) + gamma * V[s_prime])
                v += policy_func(s, a) * action_value
            V_new[s] = v
            delta = max(delta, abs(V[s] - V_new[s]))

        V = V_new.copy()

        if delta < threshold:
            print(f"✅ Converged after {iteration + 1} iterations
(Δ = {delta:.6f})")
            break

    return V

print("✅ Policy evaluation algorithm defined!")
```

```
✅ Policy evaluation algorithm defined!
```

In [3]:

```
# Run Policy Evaluation
print("🎬 Running Policy Evaluation")
print("Evaluating uniform random policy...\n")

V_evaluated = policy_evaluation_algorithm(uniform_policy, gamma=
0.9, threshold=1e-4)

print(f"\n📊 Results:")
print(f"Value at start (0,0): {V_evaluated[0]:.3f}")
print(f"Value at treasure (2,2): {V_evaluated[8]:.3f}")
print(f"Max value: {V_evaluated.max():.3f}")

# Visualize the result
plot_value_function(V_evaluated, title="Policy Evaluation Resul
t", grid_size=GRID_SIZE)
```

```
🎬 Running Policy Evaluation
Evaluating uniform random policy...

✅ Converged after 89 iterations (Δ = 0.000096)

📊 Results:
Value at start (0,0): 5.013
Value at treasure (2,2): 20.551
Max value: 20.551
```



Problems with Policy Evaluation

Curse of dimensionality: In high-dimensional state spaces, the number of possible trajectories grows exponentially. For example, a 12×150 grid yields $12^{149} \approx 6.28 \times 10^{160}$ possible paths, making comprehensive coverage for reliable value estimation infeasible.

Model requirement: The policy evaluation approach requires a transition model. In practice, we typically do not have access to this model. However, we often have data. How can we evaluate a policy based on data rather than the transition model?

[(Eysenbach, COS435 Lecture 9, 2025, p.1-3) (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-9-learning-value-functions.pdf)](https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-9-learning-value-functions.pdf)

Policy Improvement

- **Why it exists**: To systematically enhance policies by making them greedy with respect to current value estimates, ensuring progress toward optimality
- **What it computes**: Improved policies by selecting actions that maximize expected returns (Q-values) at each state
- **How it works**: Takes a value function as input and outputs a better policy by choosing the action with highest Q-value at each state

The Policy Improvement Theorem

The policy improvement step creates a new policy by acting greedily with respect to the current value function:

$$\pi'(s) = \arg\max_a \sum_{s'} p(s'|s,a)[r(s,a) + \gamma V^\pi(s')] = \arg\max_a Q^\pi(s,a)$$

Ordering on Value Functions

We define a partial order $\geq$ on value functions pointwise:

$$V_1 \geq V_2 \iff V_1(s) \geq V_2(s) \text{ for all } s \in \mathcal{S}$$

Similarly, we define $\pi' \geq \pi$ if $V^{\pi'}(s) \geq V^\pi(s)$ for all states s.

Theorem (Policy Improvement). Let π and π' be deterministic policies with π' greedy with respect to V^π:

$$Q^\pi(s, \pi'(s)) \geq V^\pi(s) \text{ for all } s \in \mathcal{S}$$

Then $\pi' \geq \pi$, i.e., $V^{\pi'}(s) \geq V^\pi(s)$ for all $s \in \mathcal{S}$.

The policy improvement theorem guarantees that each greedy step produces a policy that is at least as good as the current one, ensuring monotonic improvement toward optimality. For the complete proof, see [Sutton & Barto (2018), Section 4.2 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf)](https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf).

[(Eysenbach, COS435 Lecture 8, 2025, p.3-8) (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-8-policy-value-iteration.pdf)](https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-8-policy-value-iteration.pdf)

The Policy Improvement Algorithm

Algorithm: Policy Improvement

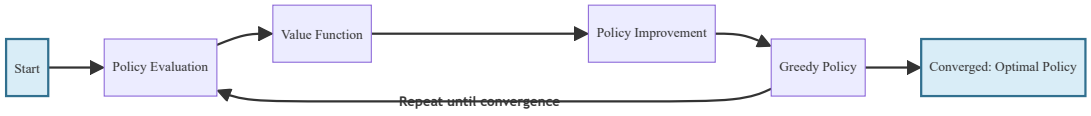
$$\textbf{Input: } V(\cdot), p(\cdot|\cdot,\cdot), r(\cdot,\cdot), \gamma, \pi(\cdot|\cdot) \tag{13}$$
$$\textbf{Output: } \pi'(\cdot), \text{policy-stable} \tag{14}$$
$$\tag{15}$$
$$\textbf{1. } \text{policy-stable} \leftarrow \text{true} \tag{16}$$
$$\textbf{2. For each state } s \in \mathcal{S}\textbf{:} \tag{17}$$
$$\textbf{3. } \text{old-action} \leftarrow \pi(s) \tag{18}$$
$$\textbf{4. } \pi'(s) \leftarrow \arg\max_a \sum_{s'} p(s'|s,a)\left[r(s,a) + \gamma V(s')\right] \tag{19}$$
$$\textbf{5. If } \text{old-action} \neq \pi'(s) \textbf{ then } \text{policy-stable} \leftarrow \text{false} \tag{20}$$
$$\textbf{6. Return } \pi', \text{policy-stable} \tag{21}$$

(Eysenbach, COS435 Lecture 8, 2025, p.6-8) (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-8-policy-value-iteration.pdf)

Policy Iteration

- **Why it exists**: To systematically find optimal policies by iteratively refining them through evaluation and improvement cycles
- **What it computes**: Optimal policies by alternating between policy evaluation (computing value functions) and policy improvement (making policies greedy)
- **How it works**: Starts with any policy, evaluates its value function completely, then improves the policy, repeating until convergence

Overview of policy iteration



Policy iteration alternates between:

1. **Policy Evaluation:** Compute the value function for the current policy
2. **Policy Improvement:** Update the policy to be greedy with respect to the value function
3. Repeat until convergence to optimal policy

(Eysenbach, COS435 Lecture 8, 2025, p.1-5) (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-8-policy-value-iteration.pdf)

The Policy Iteration Algorithm

Algorithm: Policy Iteration

$$\textbf{Input: } p(\cdot|\cdot,\cdot), r(\cdot,\cdot), \gamma \tag{22}$$
$$\textbf{Output: } \pi^*(\cdot), V^*(\cdot) \tag{23}$$
$$\tag{24}$$
$$\textbf{1. } \text{Initialize } \pi(s) \text{ arbitrarily for all } s \in \mathcal{S} \tag{25}$$
$$\textbf{2. Repeat:} \tag{26}$$
$$\quad \textbf{3. } V \leftarrow \text{PolicyEvaluation}(\pi, p, r, \gamma) \tag{27}$$
$$\quad \textbf{4. } \pi', \text{stable} \leftarrow \text{PolicyImprovement}(V, p, r, \gamma) \tag{28}$$
$$\quad \textbf{5. } \pi \leftarrow \pi' \tag{29}$$
$$\textbf{6. Until } \text{stable} = \text{true} \tag{30}$$
$$\textbf{7. Return } \pi, V \tag{31}$$

(Eysenbach, COS435 Lecture 8, 2025, p.1-5) (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-8-policy-value-iteration.pdf)

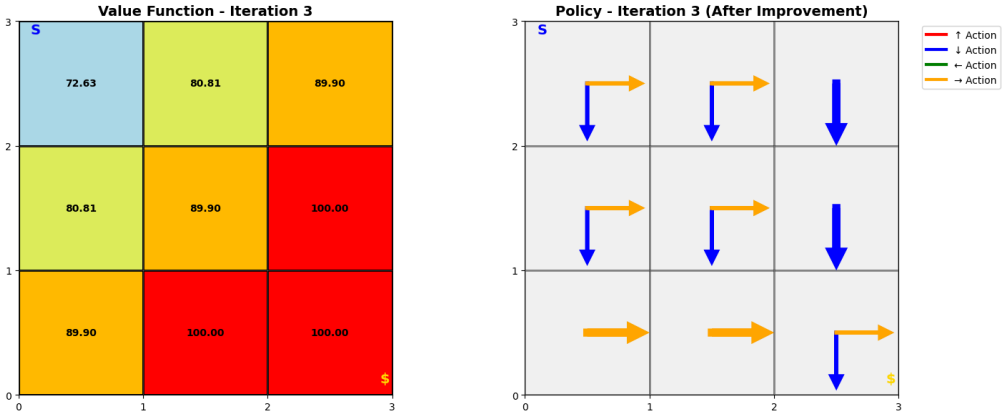
In [4]:
```
# Reimport helpers (for development)
import importlib
import helper_functions
importlib.reload(helper_functions)
from helper_functions import *

# Define policy iteration algorithm with animation (didactic inli
ne version)

def policy_iteration_algorithm(max_iterations=10):
    """
    Policy iteration: alternates between evaluation and improveme
nt.

    Returns:
        V: Converged value function
        policy_matrix: Converged policy
    """
    import time

    # Initialize uniform random policy
    policy_matrix = np.ones((NUM_STATES, NUM_ACTIONS)) / NUM_ACTI
ONS

    def policy_from_matrix(s, a):
        return policy_matrix[s, a]

    for iteration in range(1, max_iterations + 1):
        print(f"\n--- POLICY ITERATION {iteration} ---")

        # Phase 1: Policy Evaluation
        print("Phase 1: Policy Evaluation...")
        V = policy_evaluation_algorithm(policy_from_matrix, gamma
=0.9, threshold=1e-4)

        # Visualize current state
        update_combined_plot(V, policy_matrix, iteration, "(After
Evaluation)", grid_size=GRID_SIZE)
        time.sleep(1)

        # Phase 2: Policy Improvement
        print("Phase 2: Policy Improvement...")
        new_policy = np.zeros((NUM_STATES, NUM_ACTIONS))
        policy_stable = True

        for s in range(NUM_STATES):
            old_best = np.argmax(policy_matrix[s])

            # Compute Q-values
            q_values = np.zeros(NUM_ACTIONS)
            for a in range(NUM_ACTIONS):
                for s_prime in range(NUM_STATES):
                    q_values[a] += p(s_prime, s, a) * (r(s_prime)
+ 0.9 * V[s_prime])

            # Greedy policy
```

```
            best_actions = np.where(q_values == np.max(q_values))
[0]
            new_policy[s, best_actions] = 1.0 / len(best_actions)

            if np.argmax(new_policy[s]) != old_best:
                policy_stable = False

        policy_matrix = new_policy
        update_combined_plot(V, policy_matrix, iteration, "(After
Improvement)", grid_size=GRID_SIZE)
        time.sleep(1)

        print(f"Policy stable: {policy_stable}, Max value: {V.max
():.3f}")

        if policy_stable:
            print(f"\n⏳ Converged after {iteration} iteration
s!")
            break

    return V, policy_matrix

print("✅ Policy iteration algorithm defined!")
```

```
✅ Policy iteration algorithm defined!
```

In [5]:

```
# Reimport helpers (for development)
import importlib
import helper_functions
importlib.reload(helper_functions)
from helper_functions import *

# Run Policy Iteration
print("🎬 Running Policy Iteration on 3x3 Grid World")
print()

final_V, optimal_policy = policy_iteration_algorithm(max_iteratio
ns=10)

print(f"\n🎉 FINAL RESULTS:")
print(f"Optimal value function:\n{final_V.reshape(GRID_SIZE)}")
print(f"\nMaximum value: {final_V.max():.3f} at state {np.argmax
(final_V)}")

print("\n📊 Optimal Policy:")
action_names = ['↑', '↓', '←', '→']
for s in range(NUM_STATES):
    row, col = divmod(s, GRID_SIZE[1])
    best_action = np.argmax(optimal_policy[s])
    print(f"State {s} ({row},{col}): {action_names[best_actio
n]}")
```




```
Policy stable: True, Max value: 99.997

🏆 Converged after 3 iterations!

🎉 FINAL RESULTS:
Optimal value function:
[[72.62634386 80.80734386 89.89734386]
 [80.80734386 89.89734386 99.99734386]
 [89.89734386 99.99734386 99.99734386]]

Maximum value: 99.997 at state 5

📊 Optimal Policy:
State 0 (0,0): ↓
State 1 (0,1): ↓
State 2 (0,2): ↓
State 3 (1,0): ↓
State 4 (1,1): ↓
State 5 (1,2): ↓
State 6 (2,0): →
State 7 (2,1): →
State 8 (2,2): ↓
```

Convergence properties of policy iteration

Theorem (Convergence of Policy Iteration). Policy iteration converges to the optimal policy π^* in a finite number of iterations. Specifically:

1. **Monotonic Improvement**: Each policy improvement step yields $V^{\pi_{k+1}}(s) \geq V^{\pi_k}(s)$ for all states s
2. **Finite Convergence**: Since there are finitely many deterministic policies, the sequence must reach a fixed point
3. **Optimality**: At convergence, the policy π satisfies the Bellman optimality equation, making it optimal

Proof Sketch: The policy improvement theorem guarantees that each iteration produces a strictly better policy (unless already optimal). Since the MDP has a finite number of deterministic policies and we strictly improve at each step, the algorithm converges.

For the complete proof and analysis, see [Sutton & Barto (2018), Section 4.3 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf)](https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf).

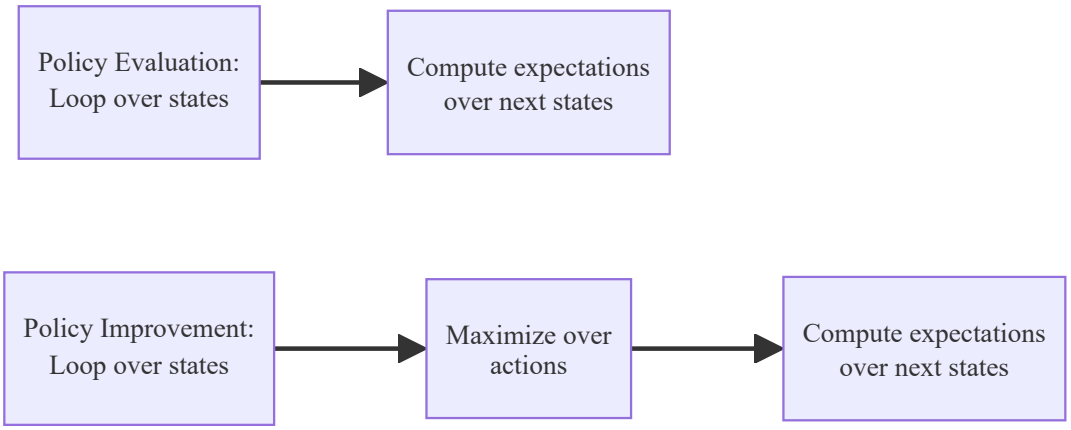
[(Eysenbach, COS435 Lecture 8, 2025, p. 3-5) (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-8-policy-value-iteration.pdf)](https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-8-policy-value-iteration.pdf)

Challenges in implementing policy iteration

Policy iteration requires access to a model of the environment to compute expected values over next states, which can be infeasible in high-dimensional or unknown environments. When using function approximation, policy iteration may not guarantee convergence due to the lack of contraction properties, leading to instability or divergence in practice.

[(Eysenbach, COS435 Lecture 8, 2025, p. 1-3) (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-8-policy-value-iteration.pdf)](https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-8-policy-value-iteration.pdf) [(Levine, CS285 Lecture 7, 2023, p. 1-28) (https://rail.eecs.berkeley.edu/deeprlcourse/deeprlcourse/static/slides/lec-7.pdf)](https://rail.eecs.berkeley.edu/deeprlcourse/deeprlcourse/static/slides/lec-7.pdf)

Time complexity of policy iteration



The time complexity of one iteration of policy iteration consists of:

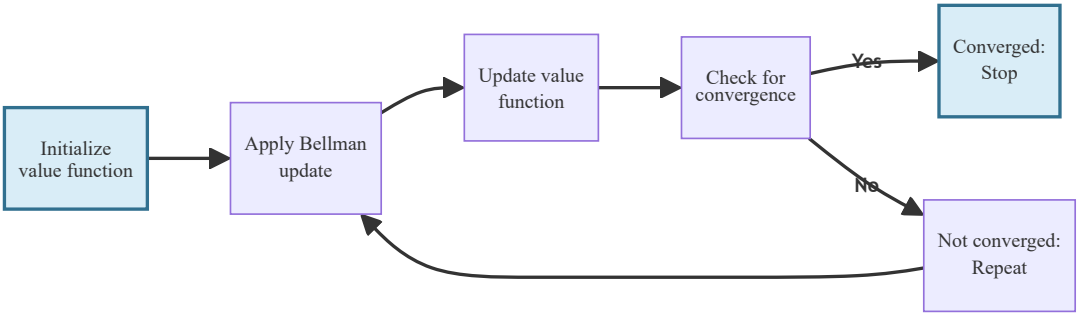
- **Policy Evaluation:** $\mathcal{O}(k|\mathcal{S}|^2|\mathcal{A}|)$ where k is the number of evaluation iterations
- **Policy Improvement:** $\mathcal{O}(|\mathcal{S}|^2|\mathcal{A}|)$ to compute the greedy policy
- **Total per iteration:** $\mathcal{O}(k|\mathcal{S}|^2|\mathcal{A}|)$ dominated by the evaluation phase

[(Eysenbach, COS435 Lecture 8, 2025, p. 1-3) (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-8-policy-value-iteration.pdf)](https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-8-policy-value-iteration.pdf)

Value Iteration

- **Why it exists**: To find optimal value functions and policies more efficiently by combining evaluation and improvement into a single update step
- **What it computes**: Optimal value functions by iteratively applying the Bellman optimality operator until convergence, then extracting optimal policies
- **How it works**: Updates value functions using the maximum over actions (Bellman optimality equation) rather than following a fixed policy

Overview of value iteration



Value iteration computes the optimal value function $V^\star(s)$ by iteratively applying the Bellman optimality equation:

$$V(s) \leftarrow \max_a [r(s,a) + \gamma \mathbb{E}_{p(s'|s,a)}[V(s')]]$$

Unlike policy iteration, value iteration doesn't maintain an explicit policy during the process but extracts the optimal policy once convergence is achieved.

[(Eysenbach, COS435 Lecture 8, 2025, p.1-3) (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-8-policy-value-iteration.pdf)](https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-8-policy-value-iteration.pdf)

The Value Iteration Algorithm (Pseudocode)

Algorithm: Value Iteration

$$\textbf{Input: } p(\cdot|\cdot,\cdot), r(\cdot,\cdot), \gamma, \theta \tag{32}$$
$$\textbf{Output: } \pi^*(\cdot), V^*(\cdot) \tag{33}$$
$$\tag{34}$$
$$\textbf{1. } \text{Initialize } V(s) = 0 \text{ for all } s \in \mathcal{S} \tag{35}$$
$$\textbf{2. Repeat:} \tag{36}$$
$$\quad \textbf{3. } \Delta \leftarrow 0 \tag{37}$$
$$\quad \textbf{4. For each state } s \in \mathcal{S}\textbf{:} \tag{38}$$
$$\quad\quad \textbf{5. } v \leftarrow V(s) \tag{39}$$
$$\quad\quad \textbf{6. } V(s) \leftarrow \max_a \sum_{s'} p(s'|s,a) \left[r(s,a) + \gamma V(s')\right] \tag{40}$$
$$\quad\quad \textbf{7. } \Delta \leftarrow \max(\Delta, |v - V(s)|) \tag{41}$$
$$\textbf{8. Until } \Delta < \theta \tag{42}$$
$$\textbf{9. Extract optimal policy:} \tag{43}$$
$$\quad \textbf{10. For each state } s \in \mathcal{S}\textbf{:} \tag{44}$$
$$\quad\quad \textbf{11. } \pi^*(s) \leftarrow \arg\max_a \sum_{s'} p(s'|s,a) \left[r(s,a) + \gamma V(s')\right] \tag{45}$$
$$\textbf{12. Return } \pi^*, V \tag{46}$$

[(Sutton & Barto, 2018, Section 4.4) (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf)](https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf) [(Eysenbach, COS435 Lecture 8, 2025, p.1-3) (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-8-policy-value-iteration.pdf)](https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-8-policy-value-iteration.pdf)

In [6]:
```
# Reimport helpers (for development)
import importlib
import helper_functions
importlib.reload(helper_functions)
from helper_functions import *

# Define value iteration algorithm (didactic inline version)

def value_iteration_algorithm(gamma=0.9, theta=0.01, max_iteratio
ns=100):
    """
    Value iteration: directly computes optimal value function.

    Returns:
        V: Optimal value function
        policy_matrix: Optimal policy extracted from V
    """
    # Step 1: Initialize V(s) = 0
    V = np.zeros(NUM_STATES)

    print("🎬 VALUE ITERATION")
    print("=" * 50)

    # Step 2-8: Iterate until convergence
    for iteration in range(1, max_iterations + 1):
        delta = 0.0
        V_new = np.zeros(NUM_STATES)

        for s in range(NUM_STATES):
            v = V[s]

            # Step 6: Bellman optimality backup
            q_values = np.zeros(NUM_ACTIONS)
            for a in range(NUM_ACTIONS):
                for s_prime in range(NUM_STATES):
                    q_values[a] += p(s_prime, s, a) * (r(s_prime)
+ gamma * V[s_prime])

            V_new[s] = np.max(q_values)
            delta = max(delta, abs(v - V_new[s]))

        V = V_new.copy()

        if iteration % 10 == 0:
            print(f"Iteration {iteration}: Δ = {delta:.6f}, max v
alue = {V.max():.3f}")

        if delta < theta:
            print(f"✅ Converged after {iteration} iterations (Δ
= {delta:.6f})")
            break

    # Steps 9-11: Extract optimal policy
    policy_matrix = np.zeros((NUM_STATES, NUM_ACTIONS))
    for s in range(NUM_STATES):
        q_values = np.zeros(NUM_ACTIONS)
```

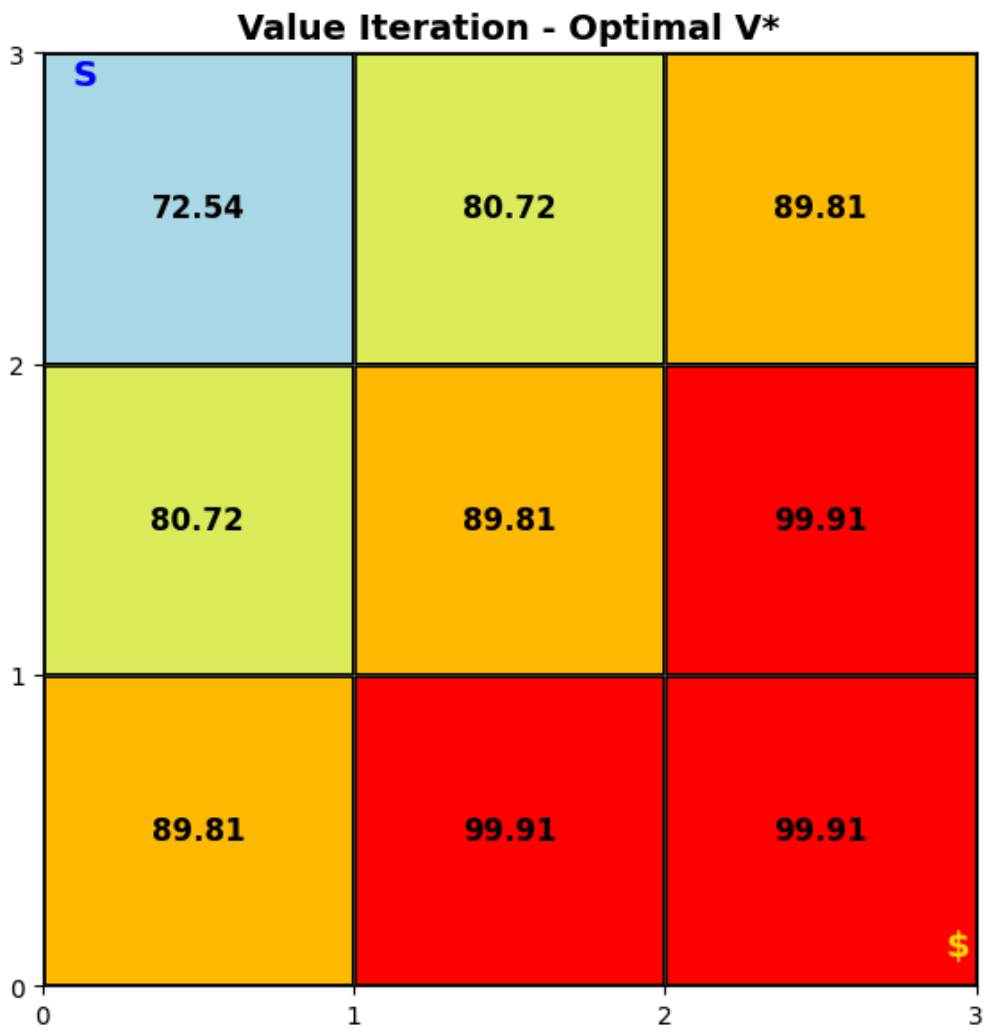
```
        for a in range(NUM_ACTIONS):
            for s_prime in range(NUM_STATES):
                q_values[a] += p(s_prime, s, a) * (r(s_prime) + g
amma * V[s_prime])

        best_actions = np.where(q_values == np.max(q_values))[0]
        policy_matrix[s, best_actions] = 1.0 / len(best_actions)

    return V, policy_matrix

print("✅ Value iteration algorithm defined!")
```

```
✅ Value iteration algorithm defined!
```

In [7]:

```
# Reimport helpers (for development)
import importlib
import helper_functions
importlib.reload(helper_functions)
from helper_functions import *

# Run Value Iteration
print("🎬 Running Value Iteration on 3x3 Grid World")
print()

V_optimal, policy_optimal = value_iteration_algorithm(gamma=0.9,
theta=0.01)

print("\n" + "=" * 50)
print("🎉 VALUE ITERATION COMPLETE!")
print("=" * 50)
print(f"\nOptimal Value Function:\n{V_optimal.reshape(GRID_SIZ
E)}")
print(f"\nMaximum value: {V_optimal.max():.3f} at state {np.argma
x(V_optimal)}")

# Visualize result
plot_value_function(V_optimal, title="Value Iteration - Optimal V
*", grid_size=GRID_SIZE)

print("\n📊 Optimal Policy:")
action_names = ['↑', '↓', '←', '→']
for s in range(NUM_STATES):
    row, col = divmod(s, GRID_SIZE[1])
    best_action = np.argmax(policy_optimal[s])
    print(f"State {s} ({row},{col}): {action_names[best_actio
n]}")
```

```
🎬 Running Value Iteration on 3x3 Grid World

🎬 VALUE ITERATION
==================================================
Iteration 10: Δ = 3.874205, max value = 65.132
Iteration 20: Δ = 1.350852, max value = 87.842
Iteration 30: Δ = 0.471013, max value = 95.761
Iteration 40: Δ = 0.164232, max value = 98.522
Iteration 50: Δ = 0.057264, max value = 99.485
Iteration 60: Δ = 0.019967, max value = 99.820
✅ Converged after 67 iterations (Δ = 0.009550)

==================================================
🎉 VALUE ITERATION COMPLETE!
==================================================

Optimal Value Function:
[[72.54304955 80.72404955 89.81404955]
 [80.72404955 89.81404955 99.91404955]
 [89.81404955 99.91404955 99.91404955]]

Maximum value: 99.914 at state 5
```



```
📊 Optimal Policy:
State 0 (0,0): ↓
State 1 (0,1): ↓
State 2 (0,2): ↓
State 3 (1,0): ↓
State 4 (1,1): ↓
State 5 (1,2): ↓
State 6 (2,0): →
State 7 (2,1): →
State 8 (2,2): ↓
```

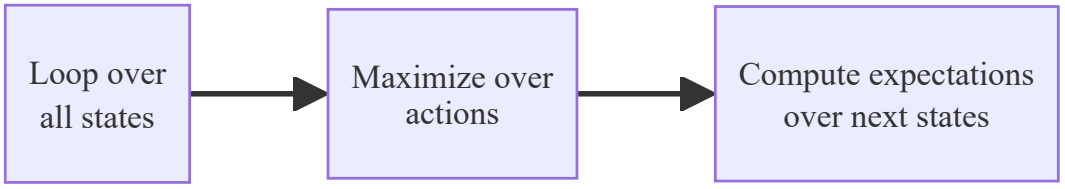
Convergence properties of value iteration

Theorem (Convergence of Value Iteration). Value iteration converges to the optimal value function V^* as the number of iterations approaches infinity. Specifically:

1. **Fixed Point Property**: The optimal value function $V^\star$ satisfies $V^\star(s) = \max_a \sum_{s'} p(s'|s,a)[r(s,a) + \gamma V^\star(s')]$ for all states s
2. **Guaranteed Convergence**: For any initial value function V_0, the sequence of value functions $V_1, V_2, \ldots$ produced by value iteration converges to V^*

See also [Sutton & Barto (2018), Section 4.4 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf)](https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf). For a proof sketch, see [(Eysenbach, COS435 Lecture 8, 2025, p. 3-5) (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-8-policy-value-iteration.pdf)](https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-8-policy-value-iteration.pdf).

Time complexity of value iteration



The time complexity of one iteration of value iteration is $\mathcal{O}(|\mathcal{S}|^2|\mathcal{A}|)$, due to:

- Looping over all states: $|\mathcal{S}|$
- Maximizing over actions: $|\mathcal{A}|$
- Computing expectations over next states: $|\mathcal{S}|$

Comparison:

- **Policy Iteration:** $\mathcal{O}(k|\mathcal{S}|^2|\mathcal{A}|)$ per iteration
- **Value Iteration:** $\mathcal{O}(|\mathcal{S}|^2|\mathcal{A}|)$ per iteration
- Trade-off: Policy iteration does more work per iteration but often converges in fewer iterations

[(Eysenbach, COS435 Lecture 8, 2025, p. 1-3) (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-8-policy-value-iteration.pdf)](https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-8-policy-value-iteration.pdf)

Summary of Dynamic Programming Methods

This lecture covered four fundamental dynamic programming methods for reinforcement learning:

1. **Policy Evaluation** - Computes the value function $V^\pi(s)$ for a given policy using the Bellman expectation operator
2. **Policy Improvement** - Creates better policies by acting greedily with respect to current value estimates
3. **Policy Iteration** - Alternates between policy evaluation and improvement until convergence to optimal policy
4. **Value Iteration** - Directly computes optimal value function $V^*(s)$ using Bellman optimality operator, then extracts policy

What's Next: Beyond Dynamic Programming

The Limitation of Dynamic Programming

While dynamic programming provides **optimal solutions** and **theoretical foundations**, it has a critical limitation:

> 📋 **Model Requirement**: Dynamic programming requires complete knowledge of the environment's **transition probabilities** $p(s'|s,a)$ and **reward function** $r(s,a)$

Real-World Challenge

In many practical applications, we don't have access to the environment model:

- **Robotics**: Unknown physics, sensor noise, environmental changes
- **Finance**: Market dynamics are complex and unpredictable
- **Game Playing**: Opponent strategies are unknown
- **Healthcare**: Patient responses vary and are difficult to model precisely

Enter Monte Carlo Methods 🎯

Next lecture introduces **Monte Carlo (MC) methods** that address this fundamental limitation:

Key Advantages of MC Methods:

✅ **Model-Free Learning**: No need for transition probabilities or reward models
✅ **Experience-Based**: Learn directly from interaction with the environment
✅ **Sample-Based Estimation**: Use actual episode outcomes to estimate value functions
✅ **Unbiased Estimates**: Provide unbiased estimates of true value functions

From Theory to Practice:

Dynamic Programming	Monte Carlo Methods
Perfect model knowledge	Learn through trial and error
Exact value functions	Statistical approximations
Guaranteed convergence	Convergence in expectation
Limited to known environments	Applicable to any environment